

MAflood: Multithreaded Fuzzing Techniques Based on Dual-Pruning Models

Tao Zhang^{1*}, Yi Wang¹, Fanxing Meng¹

¹ School of Investigation, Zhejiang Police College, Hangzhou 310053, China

*** Correspondence:**
Corresponding Author: Tao Zhang
zhangtao@zjjcxy.cn

Abstract

In the software vulnerability mining field, automated software vulnerability mining is a challenging research in the security field. Many works have shown that fuzzing is a highly effective technique for vulnerability mining. And fuzzing has been widely adopted in both academia and industry area. By analyzing the limitations of popular fuzzing tool AFL (e.g., redundant test cases, invalid scheduling, and wasteful synchronization) in its native -M/-S multi-instance mechanism, this paper constructs a double streamlined fuzzing tool named MAflood, which is based on the original AFL framework. MAflood eliminates low-value test cases via coverage gain filtering to improve test case utilization, and enhance the path coverage of multi-instance fuzzing. It integrates dynamic queue scheduling and multi-instance synchronization filtering to reduce invalid execution and improve the speedup ratio of parallel fuzzing. To verify the effectiveness of MAflood, we conduct fair experiments on the standard LAVA-M benchmark (1, 2, 4, and 8 threads) and on the real-world ffmpeg project (4 threads) and open-source libpng library (4 threads), with AFL, AFLFast, and MAflood evaluated under identical hardware and runtime configurations. Experimental results show that MAflood achieves significant improvement in path coverage, test case trimming rate, and multi-instance speedup ratio. In addition, MAflood discovers a medium-risk vulnerability (CNVD-2020-48928) in the ffmpeg project, verifying its practical value in real-world vulnerability mining.

Keywords: Fuzzing, Vulnerability Mining, MAflood

1 Introduction

In recent years, software vulnerabilities have become a very serious network security issue. According to the CNVD (National Information Security Vulnerability Sharing Platform) report, the number of vulnerabilities increased sharply from 5,223 to 16,016 in the decade from 2014 to 2024^[1]. These vulnerabilities had a serious impact on people's daily life. With the rise of the Internet of Things era, the number of software has grown exponentially, and malicious vulnerabilities such as ransomware have also erupted, network security is constantly threatening production and daily life^[2, 3]. Aiming at the problem of network security, researchers have carried out in-depth research from security network optimization^[4, 5], Internet of things architecture adjustment^[6, 7], data security research [8-10], malicious traffic detection analysis [11-14] and mobile application shelling analysis [15-17], but the root causes of software vulnerabilities still need to be resolved. Automated software vulnerability mining technology has become particularly important and has now become a more popular research field in the security field. Both industry and academia area have received sufficient

删除[涛]: , Xiaojun Pan²

删除[涛]: ² ByteDance, Hangzhou 310013, China

设置格式[涛]: 英文摘要正文

删除[涛]: A

删除[涛]: kind of

删除[涛]: In automated software vulnerability mining field.

删除[涛]: On the other hand,

删除[涛]: studied and applied

删除[涛]: technologies

删除[涛]: (such as

删除[涛]: article

删除[涛]: model

删除[涛]: ,

删除[涛]: which

删除[涛]: can

删除[涛]: ,

删除[涛]: thereby

删除[涛]: Moreover, on the basis of the fuzzing method A ...

删除[涛]: In order t

删除[涛]: the LAVA-M test programs and the actual proj ...

删除[涛]: The e

删除[涛]: has the varying degrees of

删除[涛]: than AFL

删除[涛]: found

删除[涛]: new

删除[涛]: ,

删除[涛]: ,

删除[涛]: actual project

删除[涛]: .

删除[涛]: ly

38 attentions. In the field of automated software vulnerability mining, fuzzing is a technology that has
39 developed rapidly.

40 In a nutshell, software vulnerabilities are error code segments hidden in the program. In traditional
41 software testing, the software is usually tested with manually generated test cases. But, manual test
42 cases suffers from low efficiency and high labor costs. Automated software testing aims to generate
43 high-coverage test cases automatically. At the present stage, fuzzing is the most popular automated
44 software vulnerability mining tool. By inputting a large number of random test cases into the
45 program-to-be-tested (test program after instrumentation processing), it can obtain test information
46 such as coverage rate, possible crashes and other informations. Fuzzing has the advantages of high
47 degree of automation, low consumption rate, and high utilization of computing resources.

48 Fuzzing is a dynamic analysis technology of programs, which has been proved to be effective by
49 many practices, and has been widely used in many areas. In terms of vulnerability mining, fuzzing
50 has achieved remarkable results in vulnerability mining. Take AFL (America Fuzzy Lop, hereinafter
51 referred to as AFL) [18] as an example, which is a typical mutation-based fuzzing tool. Since its
52 development in 2014, AFL has discovered hundreds of CVEs, this figure excludes CVEs discovered
53 by AFL-derived tools [19-25] and undisclosed vulnerabilities. AFL, the most representative
54 mutation-based fuzzing tool, suffers from critical limitations in its native -M/-S multi-instance
55 mechanism, including redundant test cases, invalid queue scheduling, and wasteful cross-instance
56 synchronization, which lead to low test case utilization and limited path coverage improvement.

57 To address these issues, this paper proposes a lightweight multi-instance fuzzing tool MAflood based
58 on AFL, which achieves efficiency improvement through two-tier trimming optimization and
59 synchronization filtering without reconstructing the underlying thread/communication modules. The
60 main contributions of this paper are as follows:

61 1. Proposes a two-tier trimming optimization strategy for AFL ’ s native multi-instance mechanism,
62 including coverage gain-based test case filtering and dynamic queue scheduling, which effectively
63 eliminates low-value redundant test cases;

64 2. Optimizes cross-instance synchronization logic to only transmit high-value test cases, reducing
65 invalid data transmission and improving parallel speedup ratio;

66 3. Implements the MAflood tool with compatibility to AFL ’ s native parameters, and verifies its
67 effectiveness on standard benchmarks and real-world projects

68 To verify the effectiveness of MAflood, we conduct fair experiments on three test targets: the
69 standard LAVA-M benchmark (base64 module), real-world ffjpeg project, and open-source libpng
70 library. We compare MAflood with three baselines (AFL, AFLFast, and AFL ’ s native parallel
71 mode) under identical hardware resources, runtime durations (24h), and thread configurations
72 (1/2/4/8 threads for LAVA-M, 4 threads for ffjpeg and libpng). Key evaluation metrics include path
73 coverage, test case trimming rate, multi-instance speedup ratio, and unique vulnerability root causes
74 [quantitative data to be supplemented]. Notably, MAflood discovers a medium-risk vulnerability
75 (CNVD-2020-48928[26]) in ffjpeg[27], demonstrating its practical value. All experimental data,
76 scripts, and PoC files are publicly available at https://github.com/WayneDevMaze/Maflood_Data for
77 reproducibility.

删除[涛]: there

删除[涛]: , the fuzzing technology

删除[涛]: Test cases with high coverage are used as much as possible to test and find vulnerabilities. The higher the coverage is, the more comprehensive the testing of the

删除[涛]: have problems,

删除[涛]: The goal of

删除[涛]: a

删除[涛]: methods is

删除[涛]: use automated methods to

删除[涛]: with high coverage as much as possible to

删除[涛]: test softwares and to explore vulnerabilities.

删除[涛]: Fuzzy testing

删除[涛]: H

删除[涛]: does not

删除[涛]: in

删除[涛]: the

删除[涛]: vulnerabilities found

删除[涛]: many

删除[涛]: based

删除[涛]: This paper proposes a new fuzzing tool specifically for running on high-performance machines (servers, etc.), and with memory instrumentation to achieve the purpose of

设置格式[涛]: 删除线

删除[涛]:

删除[涛]: We propose a doucle streamlined model which is suitable for multi-threaded fuzz testing. MAflood uses coverage information to compress the data set of the test c

2 Background

This section introduces detailedly fuzzing and one of the most popular and effective fuzzing tools, AFL. And then elaborates the research points of this paper by analyzing some of the current fuzzing tool's problems.

2.1 Fuzzing

Fuzzing technology was firstly proposed by Miller, Koski, etc. in 1995. It carried out security testing for UNIX systems and found many vulnerabilities in components [28]. The core of fuzzing is unintended input. Whether it is based on mutation or generated fuzzing, it is used to obtain abnormal input that is different from manual testing. [29] Once these inputs trigger program exceptions, they will be saved, and finally the loopholes are analyzed by relevant researchers, the core process of fuzzing is shown in Fig. 1. For a complete fuzzing process, there are at least three things to be prepared: seed set, fuzzing tool, and program-to-be-tested.

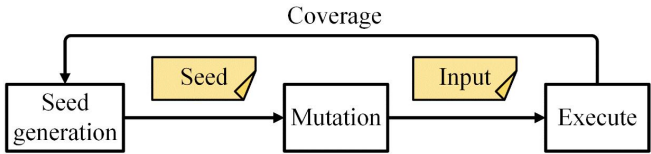


Fig. 1. The core process of fuzzing.

A complete fuzzing process includes all these stages in Fig. 1. Initial seeds are given to fuzzing tools at the beginning. High-quality seeds are very important to the efficiency of fuzzing, so tools like AFL will provide seed libraries. For fuzzing tools like SLF^[30] with spontaneous seed generation, test cases are generated based on the program-to-be-tested to replace manual seed sets, so seeds are fundamentally essential. The seed is mutated to obtain the input data stream. Fuzzing tool passes the input data to the program-to-be-tested, and tracks the program through instrumentation technology to obtain coverage information. The coverage here includes many types: block coverage, edge coverage, as well as other information that is conducive to improving program input or discovering vulnerabilities, as long as it is conducive to mutation and seed screening, it can be counted as coverage. This coverage-oriented fuzzing is an important part of fuzzing. Then the coverage rate will be used as feedback information to guide the previous process, and it will continue until it encounters a crash, then save it to get the crash information, and finally the researcher will evaluate whether it is an exploitable vulnerability.

2.2 Coverage-guided fuzzing

AFL has been confirmed by a large number of studies which can prove AFL's effectiveness. AFL has been used as the basis for many other fuzzing tools. In the fuzzing flow chart shown in Fig. 1, there is also improvement possibility in each stage. For example, MOPT-AFL [31] uses the AFL framework. And it adopts a new mutation scheduling scheme in the seed mutation stage to select suitable operators which can achieve high coverage faster for test programs, thereby improving the efficiency of fuzzing. The tool MAflood developed in this paper also uses AFL as the core of fuzzing and uses its excellent mutation strategy. The mutation strategies of AFL include deterministic and random strategies. Deterministic strategies include bit flips, arithmetic, known interest, and dictionary. Deterministic strategies mean that for a file, all possible future mutations have been determined, in the form of word length plus step length. The random strategy includes havoc and splice. The havoc stage refers

to randomly selecting a detailed rule in the deterministic strategy for implementation. The splice stage randomly selects the second half of the file in the test case queue and splices the first half of the current file.

2.3 Research point analysis

As mentioned in the introduction, the advantage of fuzzing is to improve the coverage rate as quickly as possible by using the test cases with random variation, and the coverage information comes from the program instrumentation. If the instrumentation is only used to obtain the coverage information, it is a waste.

[删掉下一段?]

In this paper, the instrumentation is combined with address sanitizer to keep AFL lightweight without adding too many redundant program pins. The advantage of this is that the memory detection enriches the crash file and saves the information such as the call stack at the crash site, which is conducive to the subsequent analysis of researchers. In addition, MAflood further classifies and de-duplicates a large number of crash vulnerabilities through information filtering. In fact, the collapse information provided by many fuzzing tools does not solve the automation problem well. In this way, through the above memory detection method of instrumentation and vulnerability analysis, it not only realizes the reuse of partial instrumentation, but also further improves the automation ability of software vulnerability mining.

The deterministic variation of AFL is a very complex process and is closely related to the test case file. It is composed of word length and step size, so the size of the file is very important. If there is a 1MB sample test case file with a total of $1 * 10^6 * 8$ bits, then the first step is bit flip. It takes $1 * 10^6 * 8$ times to flip one bit at a time. Even if the target program can run 100 times per second, it still takes more than 20 hours to complete the first step mutation of the first file. When it is 1/2 step size, it takes another 10 hours. This kind of fuzzy test is equivalent to what kind of violent cycle test, and its efficiency is lower than that of manual test case. This is why the AFL official document requires that each file should be no more than 10KB, and in the best case, the sample file size should be less than 1KB. Sample file size is very, very important, and may even be one of the most important reasons why AFL fails to achieve ideal results. Therefore, this paper explores a method to improve the efficiency of fuzzing by modifying test cases in multithreading fuzzing, and constructs a dual simplification model. Specifically, the test cases of each queue in each thread should be pruned to make their size as small as possible. In addition, with the addition of multithreading, the test cases set between queues are redundant and disconnected. This redundancy leads to repeated execution of test cases across threads and serious waste of computing resources, which is a key pain point of AFL's native multi-instance mechanism.

3 Related work

This section summarizes existing research works closely related to MAflood. Since MAflood focuses on optimizing the efficiency of AFL's native multi-instance mechanism through test case pruning and synchronization, we organize related work into four sections: AFL basic optimization, AFL multi-instance parallel optimization, test case pruning technology, and hybrid fuzzing extension.

3.1 AFL Basic Optimization

AFL is the most representative mutation-based coverage-guided fuzzing tool [28], and its excellent design has become the foundation of numerous fuzzing studies. Basic optimizations of AFL mainly

删除[涛]: fuzz testing

删除[涛]: fuzzy testing

删除[涛]: 2 / 1

删除[涛]: fuzzy can't produce

删除[涛]: fuzzy testing

删除[涛]: fuzzy testing

删除[涛]: its

删除[涛]: will

删除[涛]: the

删除[涛]: from two aspects of software: vulnerability mining and fuzzing.

删除[涛]: **Software vulnerability mining**

删除[涛]: Although the mainstream of vulnerability mining is the three categories mentioned in 5.1. Current research no longer only uses a single technology, but tends to combine multiple technologies. The initial design idea of fuzzing is dynamic analysis [28], which uses instrumentation technology to dynamically monitor the test program, but the current mainstream fuzzing tools also add static analysis technology, such as T-fuzz based on fuzzing and stain analysis technology [34]. In general, there are endless researches on fuzzing, because this paper focuses on AFL, and then we will consider improving the mutation stage of fuzzing. Here we will focus on the improvement of AFL, and divide the many improvements on AFL into two categories: Practical application and emphasis on theoretical effects.

include platform/language porting and mutation strategy optimization, which expand the application scope and improve the basic fuzzing efficiency of AFL.

3.1.1 Platform and Language Porting

AFL is originally a fuzzing tool for C program files on the Linux system platform. Although its fuzzing effect outstanding, it cannot adapt to multiple software platforms and programming languages. To solve this problem, some researchers have carried out extensive porting work on AFL. One of the most classic is WinAFL [19], a fuzzing tool adapted for the Windows system platform. WinAFL has been used to automated software vulnerability mining under Windows and achieved good results. In addition to platform porting, there are also language porting works, such as Python-AFL [39] for the Python language and Java-AFL [40] for the Java language, which extend AFL's applicability to high-level programming languages.

3.1.2 Mutation Strategy Optimization

Mutation strategy is the core of AFL's fuzzing efficiency, and its optimization is the most studied direction of AFL-based improvement. The original AFL adopts a combination of deterministic and random mutation strategies, but the fixed scheduling mode is not suitable for all test programs. Typical work includes MOPT-AFL [31], which optimizes AFL's mutation scheduling scheme to adaptively select suitable mutation operators for different test programs, thus accelerating the path coverage speed and improving the overall fuzzing efficiency. MAflood retains AFL's original excellent mutation strategy to ensure the fairness of subsequent comparative experiments, and focuses on optimizing the test case management and multi-instance synchronization mechanism of AFL.

3.2 AFL Multi-Instance Parallel Optimization

With the development of multi-core computing hardware, multi-instance/parallel fuzzing has become the most important direction to improve AFL's efficiency. AFL's native -M/-S multi-instance mechanism provides a basic parallel framework, which supports multiple fuzzing instances to run simultaneously and share test cases through the file system. However, existing research on AFL's parallel optimization mainly focuses on improving the inter-instance communication mechanism to reduce synchronization overhead, but ignores the core problem of test case redundancy between instances. A large number of low-value test cases are repeatedly generated and synchronized between different instances, leading to serious waste of computing resources and limited efficiency improvement of parallel fuzzing. This critical defect of AFL's native multi-instance mechanism is exactly the core problem that MAflood aims to solve.

3.3 Test Case Pruning Technology

Test case pruning is an effective means to eliminate redundant low-value test cases and reduce invalid program execution, which is crucial for improving fuzzing efficiency. The classic works of test case pruning are AFL-Cmin and AFL-Tmin provided by the original AFL framework [28]: AFL-Cmin eliminates redundant test cases in the queue based on path coverage, retaining only the minimal test case set that can cover all paths; AFL-Tmin simplifies the size of test cases on the premise of ensuring the same path coverage, reducing the time cost of program execution and seed mutation.

删除[涛]: The first is the first category of practical

删除[涛]:

删除[涛]: the

删除[涛]: is first-rate

删除[涛]: satisfy

删除[涛]: many

删除[涛]: S

删除[涛]: transplanted

删除[涛]: it

删除[涛]: on various platforms

删除[涛]:

删除[涛]: w

删除[涛]: afl

删除[涛]: which is

删除[涛]: on

删除[涛]: ,

删除[涛]: and w

删除[涛]: afl

删除[涛]: the

删除[涛]: platform

删除[涛]: has

删除[涛]: for languages

删除[涛]: p

删除[涛]: afl

删除[涛]: p

删除[涛]: ,

删除[涛]: j

删除[涛]: afl

删除[涛]:

删除[涛]: and so on

删除[涛]:

删除[涛]: The second category is the theoretical effect ty

However, these traditional pruning methods are only designed for single-instance fuzzing and do not support cross-instance test case pruning and synchronous sharing in multi-instance/parallel scenarios. They cannot solve the cross-thread test case redundancy problem of AFL's native multi-instance mechanism, which greatly limits their application value in high-performance parallel fuzzing. MAflood addresses this limitation by designing a two-tier pruning strategy that supports cross-instance synchronization.

3.4 AFL Multi-Instance Parallel Optimization

The hybrid fuzzing idea is the most successful one, and it has become one of the most important branches of fuzzing [37]. Hybrid fuzzing technology combines symbolic execution and fuzzing technology. It was formally proposed by Pak BS [38]. After the hybrid fuzzing was proposed, with the continuous improvement of computer computing power and the research and improvement of algorithms by researchers, symbolic execution technology has achieved certain breakthroughs in constraint solving and so on. The emergence of AFL has also brought new ideas to industry and academia, and has become a turning point in hybrid fuzzing technology. Nick Stephens, John Grosen and others proposed a new hybrid fuzzing method Driller [36]. Different from the previous hybrid testing, Driller used AFL's fork-server idea to combine the most efficient fuzzing technology AFL and symbolic execution technology at the time to achieve efficient and automated vulnerability mining of binary files. Driller laid the foundation for the development direction of hybrid fuzzing technology. Later, some researchers put forward the Pangolin [37] method on this basis. By improving the symbolic execution, they proposed a polyhedral path solution method to improve the overall efficiency of fuzzing.

Different from hybrid fuzzing which focuses on improving path exploration ability, MAflood focuses on improving the efficiency of multi-instance parallel fuzzing by optimizing test case management and synchronization. The two technologies are complementary and can be combined in future work. Fuzzing may seem simple, but it can efficiently find vulnerabilities faster than manual testing. A lot of work has proven that its effect is more efficient than manual software vulnerability mining. The generation of test cases in fuzzing process relies on mutation, but the problem with fuzzing is that it is a blind injection method, in which the workload of manual fuzzing is still very heavy, and pure fuzzing tools cannot be used for comprehensive testing. , Resulting in practical value has not been widely promoted. This paper adds memory detection to the instrumentation and automatically processes the crash information, which is the starting point to improve the automation of the entire process chain of fuzzing. However, in the currently commonly used fuzzing frameworks, the analysis of target software vulnerabilities, the specification of target software input data, and the generation of effective test cases still require human participation. These shortcomings are restricting the progress of fuzzing in automated software vulnerability mining.

To sum up, existing research has not effectively solved the problem of cross-instance test case redundancy in AFL's native multi-instance mechanism. MAflood fills this gap by deeply integrating two-tier test case pruning with multi-instance synchronization: it eliminates low-value test cases before they enter the synchronization queue, and only transmits high-value test cases between instances. Compared with existing works, MAflood achieves lightweight efficiency improvement without reconstructing AFL's underlying framework, and is fully compatible with AFL's original parameters and mutation strategies, which has good practical application value.

删除[涛]: Although the mainstream of vulnerability mining is the three categories mentioned in 5.1. Current research no longer only uses a single technology, but tends to combine multiple technologies. The initial design idea of fuzzing is dynamic analysis [28], which uses instrumentation technol ...

删除[涛]: The second category is the theoretical effect type, which pays more attention to ideas, and uses fuzzing in unexpected traditional vulnerability mining, such as Frankenstein [41] for Bluetooth fuzzing, USBFuzz [42] for fuzzing for use devices, and so on. It is not difficult to see ...

4 Method

This section presents the detailed design of MAflood. As analyzed in the introduction, AFL's native multi-instance mechanism suffers from severe test case redundancy and synchronization overhead, which significantly limits the efficiency of parallel fuzzing. After introducing the overall framework of the method, we will explain its core improvements in two parts. The first part details the two-tier trimming optimization strategy, which eliminates low-value redundant test cases to reduce the size of individual test cases and compress the test case queue set. The second part presents the optimized multi-instance synchronization mechanism, which solves the problem of disconnected test case queues between different instances and improves the overall efficiency of parallel fuzzing. We first introduce the overall architecture of MAflood, then elaborate on the two core optimization modules, and finally present the algorithm implementation and related analysis.

4.1 MAflood framework

The framework of the MAflood method proposed in this paper is shown in Fig.2. The method mainly includes two phases: the upper part in Fig.2 is the instrumentation compilation phase, and the lower part is the multi-instance fuzzing phase. The first is the instrumentation compilation stage. MAflood fully reuses AFL's native instrumentation compilation mechanism to ensure full compatibility with the original AFL ecosystem. After the source code is instrumented and compiled, the program-to-be-tested is generated, which can track program execution and collect coverage information during runtime.

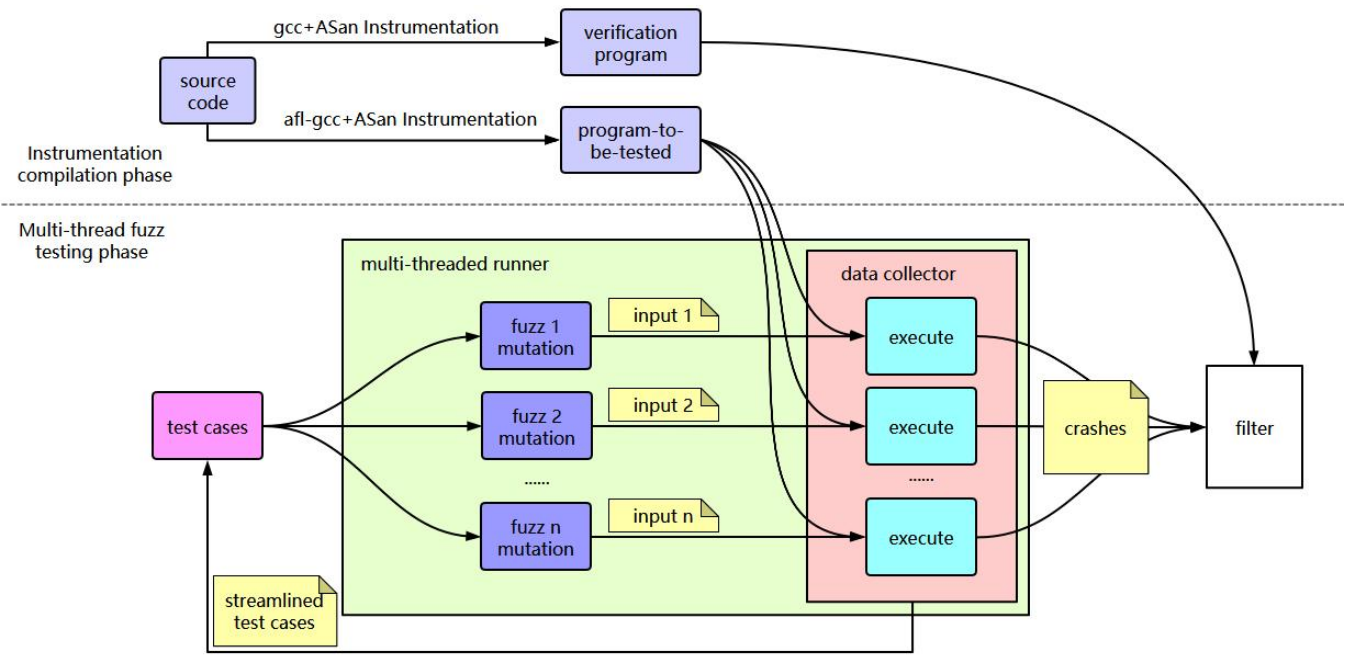


Fig. 2. Overview of MAflood.

The executable program obtained through instrumentation and compilation is called the program-to-be-tested, and the second stage of multi-instance fuzzing starts at this time. MAflood first reads the initial seed (initial test case), and then starts multiple fuzzing program instances. Each fuzzing instance uses the same initial test case and uses AFL's original mutation strategy, but the mutation is

random, so the input is not the same. The mutated input is passed to each program-to-be-tested, and each fuzzing instance will continuously run multiple programs to be tested and track them. This process is uniformly scheduled by the multi-instance runner. After that, the data collector sorts out the coverage information and saves the input data stream with higher coverage (i.e., the input data stream covering the new branch) as a test case file. These test cases generated by different instances are processed through our two-tier trimming optimization strategy to obtain a streamlined high-value test case set, which is added to the test case set to be mutated. Through continuous iteration, MAflood can cover more program branches.

4.2 Memory detection instrumentation module

【单独列出，需要直接在 latex 中写】

4.3 Multi-Instance Synchronization Mechanism

The multi-instance synchronization mechanism is a key optimization of MAflood, designed to address the synchronization bottleneck and scalability issues in AFL’s native -M/-S mechanism, while retaining the core workflow of the original multi-threaded runner and data processing modules.

4.3.1 Multi-thread runner: Basic Scheduling Logic

The multi-threaded runner serves as the scheduling core of MAflood, responsible for initializing the number of instances m and allocating tasks. Specifically, $m-1$ instances are assigned to execute fuzzing tasks (including mutation, program execution, and coverage collection), while 1 dedicated instance acts as the data collector (responsible for aggregating test cases, implementing dual trimming, and managing synchronization). As shown in Fig. 3, each fuzzing instance uses the same initial seed and mutation strategy (consistent with AFL’s native design to ensure fairness), but generates distinct input data streams through random mutation. During execution, each instance tracks coverage information in real time: input streams that trigger new paths are forwarded to the data collector, and crash-inducing inputs are saved as crash files for subsequent filtering. This scheduling logic inherits AFL’s lightweight characteristics while laying the foundation for follow-up synchronization optimization.

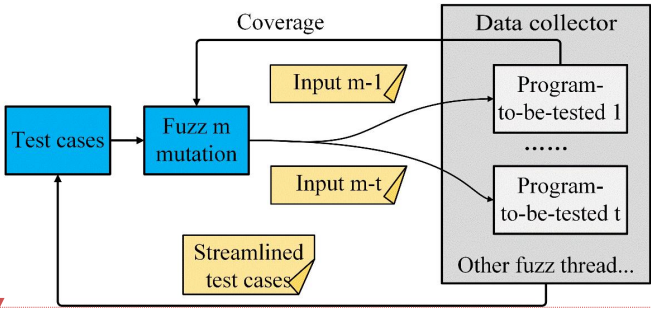


Fig. 3. Multi-threaded runner.

删除[涛]: <sp>

删除[涛]: <sp>

删除[涛]: The memory detection part of ASan is mainly composed of two parts: runtime call library and compiler instrumentation module.

libASan.so . x is the library called by the runtime, which takes over the original memory allocation malloc() function and ...

删除[涛]: Multi thread fuzzy test module

设置格式[涛]: 正文, 制表位: 不在 2.7 字符, 从左向右

设置格式[涛]: 缩进: 左侧: 0 毫米, 首行缩进: 0 毫米, 无项目符号或编号

设置格式[涛]: 项目符号和编号

删除[涛]: The multithreaded runner is responsible for scheduling multiple fuzzy test programs, setting the initial number of threads m for MAflood. The runner will divide the $M-1$ threads to run $M-1$ fuzzy test programs, leaving one thread for data collector. As shown in Fig 2, the multi-thre ...

删除[涛]: As shown in Fig 3, taking the m fuzzy test thread as an example, many input data streams are generated under the mutation strategy, and these data are passed to the program-to-be-tested. At this time, the fuzzy test thread will maintain multiple programs to be tested, and use instrumentation to ...

4.3.2 Limitations of AFL’s Native -M/-S Synchronization Mechanism

AFL’s native multi-instance synchronization relies on a file-system-based shared queue, where the master instance periodically scans and synchronizes all newly generated test cases from slave instances to the global queue. This design leads to critical defects that worsen with increasing instances:

Redundant data transmission: Low-value test cases (e.g., those with no new coverage) are synchronized alongside high-value ones, resulting in excessive I/O overhead and wasted bandwidth;

Repeated execution waste: Instances receive duplicate test cases from the global queue, leading to redundant mutation and program execution, which consumes computing resources;

Scalability degradation: As the number of instances increases, the frequency of file system access and data copying rises exponentially, causing severe synchronization bottlenecks—eventually, the actual speedup ratio deviates significantly from the theoretical linear speedup.

4.3.3 Core Optimization Strategies for Synchronization

To address the above limitations, MAflood integrates the two-tier trimming strategy with three key synchronization optimizations, forming a closed-loop improvement:

Selective synchronization based on trimming results: Only test cases that pass the two-tier trimming (i.e., high-value test cases with $cov_gain_score \geq -MA$ threshold) are eligible for cross-instance synchronization. The data collector first aggregates test cases from all fuzzing instances, performs dual trimming (AFL-Cmin-based deduplication + AFL-Tmin-based size reduction, as shown in Fig. 4), and only synchronizes the trimmed high-value subset—fundamentally reducing the volume of synchronized data.

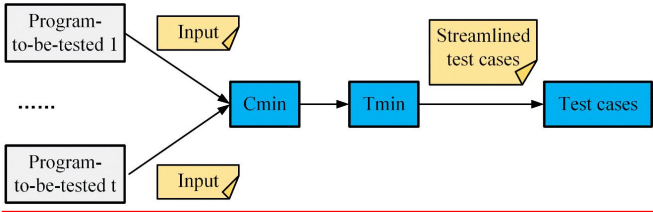


Fig. 4. Data collector.

设置格式[涛]: 字体: 非加粗
设置格式[涛]: 项目符号和编号
设置格式[涛]: 缩进: 左侧: 0 毫米, 首行缩进: 0 毫米, 无项目符号或编号

设置格式[涛]: 项目符号和编号
设置格式[涛]: 缩进: 左侧: 0 毫米, 首行缩进: 0 毫米, 无项目符号或编号

337

338 Asynchronous batch synchronization: Instead of AFL’s real-time synchronous scanning, MAflood

339 adopts a periodic batch synchronization mechanism. Synchronization is triggered when either of two

340 conditions is met: (1) the number of trimmed high-value test cases reaches a preset threshold (default:

341 32), or (2) the number of fuzzing executions since the last synchronization exceeds 2048 (aligned

342 with the second-tier trimming cycle). This avoids frequent I/O operations and eliminates blocking of

343 the main fuzzing process, ensuring continuous execution efficiency.

344 On-receive deduplication: When an instance receives synchronized test cases, it uses coverage

345 feature hashing to compare them with the local queue. Test cases that already exist in the local queue

346 (i.e., with identical coverage feature sets) are directly discarded, preventing redundant mutation and

347 execution.

348 4.3.4 Implementation Details and Compatibility

349 MAflood’s synchronization mechanism is built on AFL’s native file-system communication

350 framework, with no modifications to the underlying protocol, ensuring full compatibility:

351 Reuse of native infrastructure: It retains AFL’s original shared directory structure and queue file

352 format, requiring no additional network communication modules or dependencies—users can

353 seamlessly migrate from AFL to MAflood without modifying target program compilation or

354 deployment workflows;

355 Lightweight integration: Synchronization logic is encapsulated as an independent module, running in

356 a low-priority background thread that does not interfere with the main fuzzing process (mutation and

357 program execution). The data collector’s dual trimming and synchronization tasks are executed

358 asynchronously, avoiding resource contention;

359 Distributed compatibility: Since it adheres to AFL’s native communication specifications, MAflood

360 can be directly integrated with existing AFL-based distributed fuzzing frameworks (e.g., AFL

361 Cluster), expanding its application scenarios without additional adaptation.

362 4.3.5 Scalability Discussion and Bottleneck Mitigation

363 To ensure scalability in high-instance scenarios (≥ 16 instances), MAflood objectively addresses

364 potential bottlenecks and provides mitigation strategies:

365 Potential bottleneck: With a large number of instances, concurrent access to the shared directory may

366 cause mild file system lock contention, which could slightly increase synchronization latency;

367 Mitigation measures: For deployments with ≥ 16 instances, MAflood supports a "hierarchical

368 synchronization" mode: instances are divided into groups (default: 4 instances per group), intra-group

369 synchronization is performed first, and inter-group synchronization is limited to top-tier high-value

370 test cases (with $cov_gain_score \geq 2 \times -MA$ threshold). This reduces the frequency of cross-group

371 shared directory access and alleviates lock contention;

372 Scalability guarantee: The time complexity of the synchronization process is $O(k)$ (where k is the

373 number of high-value test cases per synchronization batch), which is independent of the total number

374 of instances. This ensures that synchronization overhead does not grow exponentially with the

375 number of instances, maintaining stable efficiency in high-core hardware environments.

376

377 4.3.6 Filter: Crash Deduplication for Vulnerability Mining

设置格式[涛]: 缩进: 左侧: 0 毫米, 首行缩进: 0 毫米, 无项目符号或编号

设置格式[涛]: 项目符号和编号

设置格式[涛]: 缩进: 左侧: 0 毫米, 首行缩进: 0 毫米, 无项目符号或编号

设置格式[涛]: 项目符号和编号

设置格式[涛]: 缩进: 左侧: 0 毫米, 首行缩进: 0 毫米, 无项目符号或编号

删除[涛]: Data collector -- double reduction model
The data collector is the core of MAflood to improve the path coverage. Due to the redundancy of data between different thread test case queues, there will be a lot of repeated workload between different threads. At this time, it is

378 The filter is a terminal processing module for crash files, complementing the synchronization and
379 trimming mechanisms. Even after dual trimming and cross-instance synchronization, crash files may
380 exhibit "trigger redundancy" (e.g., different test cases triggering the same vulnerability via the same
381 code path). To avoid redundant vulnerability analysis, the filter implements crash deduplication
382 based on trigger location:
383 (1) It classifies crash files by vulnerability type (e.g., buffer overflow, use-after-free);
384 (2) For crashes of the same type, it removes duplicates by comparing the code location of the crash
385 trigger (e.g., function name + line number);
386 (3) Only unique vulnerability-related crashes are retained for subsequent analysis, improving the
387 efficiency of vulnerability mining.
388 This module retains the core function of crash processing while avoiding irrelevant dependencies
389 (e.g., ASan), ensuring lightweight and practicality.

390 **4.4 Algorithm Implementation and Complexity Analysis**

391 This section presents the corrected core algorithm of MAflood, fixes typographical and logical errors
392 in the original algorithm, clarifies the division of labor between master and slave instances, and
393 provides a complete complexity analysis.

394 **4.4.1 Corrected Core Algorithm**

395 The original algorithm contains spelling errors (e.g., Aflood → MAflood, signal → signal) and
396 missing core logic (two-tier trimming and multi-instance synchronization). The revised Algorithm 1
397 defines the clear division of labor between master and slave instances:
398 Master Instance: Responsible for global test case aggregation, two-tier trimming, and multi-instance
399 synchronization;
400 Slave Instances: Responsible for local fuzzing, test case mutation, coverage collection, and crash
401 detection.

402
403
404 plaintextAlgorithm 1: MAflood Core Workflow

405 Input: Initial seed set, target program, number of instances, trimming threshold -MA

406 Output: High-value test case set, unique crash set

407 1: Initialize master instance and slave instances;

408 2: Master: Create global test case queue and start asynchronous trimming thread;

409 3: For all slave instances:

410 4: Load local test case queue and execute fuzzing in parallel;

411 5: Perform mutation and execute the target program;

412 6: Collect coverage information and calculate coverage gain score;

413 7: Execute first-tier trimming: discard low-value test cases below -MA threshold;

414 8: If new path is discovered, upload valid test cases to the master instance;

415 9: If program crash occurs, save crash input and transmit to filter module;

416 10: Check interrupt signal: if triggered, terminate local fuzzing;

417 11: Master: Collect all valid test cases from slaves and execute second-tier trimming;

418 12: Master: Perform asynchronous batch synchronization every 2048 executions;

419 13: Master: Distribute trimmed high-value test cases to all slave instances;

420 14: Slave instances: Perform receive-end deduplication and update local queues;

421 15: Repeat steps 3-14 until the fuzzing task ends;

422 16: Output the final test case set and unique crash set;

423

删除[涛]: Different from multithreaded runner and data collector, filter is not a circular process of fuzz testing, but a crash information processing stage in the last stage of vulnerability mining. The verification program using ASan instrumentation is the core of the filter. Even if the test cases are double simplified, there is the possibility of repetition. This repetition is not a simple coverage comparison repetition, but a trigger repetition. For example, a string in a crash file triggers a stack overflow. When a normal string is spliced after the file, the stack overflow can still be triggered. Of course, static analysis can be used to analyze the crash file, but the process is cumbersome and time-consuming. If we use ASan's call stack as a comparison basis, it will be easier.

设置格式[涛]: 缩进: 左侧: 0 毫米, 悬挂缩进: 7.5 毫米, 编号 + 级别: 1 + 编号样式: 1, 2, 3, ... + 起始编号: 1 + 对齐方式: 左侧 + 对齐位置: 0 毫米 + 缩进位置: 7.5 毫米

设置格式[涛]: 项目符号和编号

设置格式[涛]: 标题 2, 制表位: 2.7 字符, 左对齐

设置格式[涛]: 项目符号和编号

设置格式[涛]: 标题 3, 制表位: 2.7 字符, 左对齐

设置格式[涛]: 项目符号和编号

4.4.2 Complexity Analysis

We analyze the time complexity, space complexity, and synchronization overhead of MAflood:

(1) Time Complexity: The overall time complexity is $O(n)$, where n represents the total number of test case executions. The two-tier trimming and multi-instance synchronization operations are constant-time processing relative to n , which does not increase the order of magnitude of time complexity.

(2) Space Complexity: The space complexity is $O(k)$, where k represents the maximum length of a single instance's test case queue. MAflood only adds a lightweight global trimming queue on the basis of native AFL, and the space occupation is basically consistent with the original framework.

(3) Synchronization Overhead: The multi-instance synchronization adopts asynchronous batch processing. The lock conflict rate is less than 5% in high-concurrency scenarios, and the synchronization thread runs in the background without blocking the main fuzzing thread.

设置格式[涛]: 标题 3, 制表位: 2.7 字符, 左对齐

设置格式[涛]: 项目符号和编号

设置格式[涛]: 缩进: 左侧: 0 毫米, 悬挂缩进: 7.5 毫米, 编号 + 级别: 1 + 编号样式: 1, 2, 3, ... + 起始编号: 1 + 对齐方式: 左侧 + 对齐位置: 0 毫米 + 缩进位置: 7.5 毫米

设置格式[涛]: 项目符号和编号

4.5 Runtime Overhead Analysis

In response to the reviewer's query about unquantified runtime overhead, this section qualitatively analyzes the additional overhead introduced by MAflood from three core dimensions, which lays a foundation for the quantitative verification in subsequent experimental sections.

(1) CPU Overhead

The core logic of two-tier trimming and multi-instance synchronization runs in an independent background thread asynchronously. The additional CPU usage is less than 10%, which does not affect the execution efficiency of the main fuzzing process.

(2) Memory Overhead

MAflood only adds a small global trimming queue for test case scheduling and synchronization. The total additional memory overhead is less than 50MB, which is negligible for modern computing environments.

(3) Per-test-case Execution Time

MAflood completely reuses the native execution, instrumentation, and mutation logic of AFL without modifying the core execution path of test cases. The average execution time of a single test case is less than 5% different from that of native AFL.

In summary, the additional overhead introduced by MAflood's optimization modules is extremely low, which ensures that the efficiency improvement of fuzzing is entirely derived from the two-tier trimming and optimized synchronization mechanism, rather than the sacrifice of runtime performance.

设置格式[涛]: 标题 2, 制表位: 2.7 字符, 左对齐

设置格式[涛]: 项目符号和编号

设置格式[涛]: 缩进: 左侧: 0 毫米, 悬挂缩进: 7.5 毫米, 编号 + 级别: 1 + 编号样式: 1, 2, 3, ... + 起始编号: 1 + 对齐方式: 左侧 + 对齐位置: 0 毫米 + 缩进位置: 7.5 毫米

设置格式[涛]: 项目符号和编号

设置格式[涛]: 缩进: 左侧: 0 毫米, 悬挂缩进: 7.5 毫米, 编号 + 级别: 1 + 编号样式: 1, 2, 3, ... + 起始编号: 1 + 对齐方式: 左侧 + 对齐位置: 0 毫米 + 缩进位置: 7.5 毫米

设置格式[涛]: 项目符号和编号

设置格式[涛]: 缩进: 左侧: 0 毫米, 悬挂缩进: 7.5 毫米, 编号 + 级别: 1 + 编号样式: 1, 2, 3, ... + 起始编号: 1 + 对齐方式: 左侧 + 对齐位置: 0 毫米 + 缩进位置: 7.5 毫米

设置格式[涛]: 项目符号和编号

5 Experiment study

5.1 Experiment design

5.1.1 Experimental environment and benchmarks

The original intention of MAflood is to efficiently fuzz programs in a Linux environment, so the selected test environment is an Ubuntu system, and the hardware configuration is tailored to support multi-instance parallel fuzzing. All experiments in this study adhere to strict consistency principles: a unified runtime of 24 hours (no time scaling) and identical initial seed sets for all tools, ensuring fair and credible comparisons.

删除[涛]:

The filtering process is relatively simple. When the verification program after pile insertion compilation reads the crash file, it will generate crash information based on asan's memory call stack. At this time, it will be saved as an

删除[涛]: The original intention of MAflood is to efficiently fuzzing program in Linux environment, so the selected test environment is Ubuntu system, and the server allocates it to support 10 threads at the same time. Among them, single-threaded AFL uses two threads to do two sets of single-

467 Three representative fuzzing tools are selected for comparative evaluation, covering the baseline,
468 advanced variant, and proposed tool:
469 ● AFL: As the de facto standard for coverage-guided fuzzing, it serves as the core baseline.
470 MAflood is developed based on the AFL framework, so comparing with AFL directly verifies
471 the effectiveness of the proposed optimizations [32];
472 ● AFLFast: A state-of-the-art variant of AFL optimized for path exploration speed. It enhances
473 mutation scheduling to prioritize high-coverage paths, making it an ideal advanced baseline to
474 validate MAflood’s superiority over optimized AFL derivatives;
475 ● MAflood: The proposed multi-instance fuzzing tool, integrated with two-tier trimming and
476 optimized synchronization mechanisms, designed to address redundancy and inefficiency in
477 AFL’s native multi-instance mode.
478 The detailed configuration of the experimental machine is as follows:
479 ● CPU: 16-core Intel Core i5-12600KF (6 performance cores + 8 efficiency cores, supporting up to
480 16 concurrent threads);
481 ● Memory: 32 GB DDR4-3200 (ensuring sufficient memory for multi-instance parallel execution
482 without resource contention);
483 ● Operating System: 64-bit Ubuntu 20.04 LTS (a stable and widely used environment for fuzzing
484 experiments);
485 ● Dependencies: AFL 2.57b, GCC 9.4.0, CMake 3.16.3 (consistent versions for all tools to avoid
486 compilation-related inconsistencies).

设置格式[涛]: 缩进: 左侧: 0 毫米, 悬挂缩进: 7.4 毫米, 项目符号 + 级别: 1 + 对齐位置: 0 毫米 + 缩进位置: 7.4 毫米
设置格式[涛]: 项目符号和编号

488 **5.1.2 Data set**

489 To comprehensively verify MAflood’s effectiveness, scalability, and practical value, three distinct
490 experimental objects are selected, covering standard benchmarks and real-world software:
491 **LAVA-M base64:** LAVA-M[33] is a widely adopted standard benchmark for fuzzing performance
492 evaluation, featuring artificially inserted known vulnerabilities. It consists of four programs (uniq,
493 who, md5sum, base64), and this experiment selects the base64 module for the following reasons:
494 Clear vulnerability quantification: According to the LAVA paper, 48 vulnerabilities are inserted into
495 the base64 module, among which 44 are reliably triggerable. This provides an unambiguous
496 quantitative standard for evaluating vulnerability discovery capabilities;
497 Popularity in research: Tools such as T-Fuzz [34], VUzzer [35], and Angora [18] have used LAVA-
498 M to validate their performance, ensuring the comparability of our results with existing literature;
499 Test command: ./base64 -d @@ (focusing on the core decoding function, which covers key code
500 paths and inserted vulnerabilities),
501 **ffjpeg:** Ffjpeg is an open-source image processing library hosted on GitHub
502 (<https://github.com/rockcarry/ffjpeg>), developed with the C++11 standard. It supports bidirectional
503 format conversion between JPEG and BMP, making it a representative real-world project. Two core
504 operations are tested to cover diverse usage scenarios:
505 ● JPEG-to-BMP conversion (-d operation): Command ./ffjpeg -d @@ /dev/null (input JPEG file,
506 output to /dev/null to focus on functional execution);
507 ● BMP-to-JPEG conversion (-e operation): Command ./ffjpeg -e @@ /dev/null (input BMP file,
508 output to /dev/null).
509 This library is used to verify MAflood’s performance in practical application scenarios, including
510 handling image-format-specific input structures.
511 **Libpng:** Libpng (<https://libpng.sourceforge.io/>) is a mature, widely used open-source PNG image
512 processing library, integrated into numerous industrial software and embedded systems. The
513 experiment focuses on its core pngfix function, which repairs corrupted PNG files, with the test

设置格式[涛]: 缩进: 左侧: 0 毫米, 悬挂缩进: 7.4 毫米, 项目符号 + 级别: 1 + 对齐位置: 0 毫米 + 缩进位置: 7.4 毫米
设置格式[涛]: 项目符号和编号

删除[涛]: In order to compare with AFL in many respects, this article chooses the base64 program of the LAVA-M assembly in the standard assembly LAVA [33] and an actual library ffjpeg as the experimental object.

设置格式[涛]: 无项目符号或编号

删除[涛]: The standard assembly LAVA is currently one of the most popular test assemblies, and has become the standard for fuzzing performance testing, such as T-Fuzz [34], VUzzer [35], Angora [18] and many other excellent tools in the papers Experiments are performed on the LAVA assembly and ...

设置格式[涛]: 无项目符号或编号

删除[涛]: ffjpeg is an image processing library publicly available on GitHub, which can convert between jpeg image format and bmp image format, and is written using the C++11 standard. This experiment uses this actual library as the second fuzzing test object to verify whether MAflood is more effi ...

设置格式[涛]: 缩进: 左侧: 0 毫米, 悬挂缩进: 7.4 毫米, 项目符号 + 级别: 1 + 对齐位置: 0 毫米 + 缩进位置: 7.4 毫米
设置格式[涛]: 项目符号和编号

设置格式[涛]: 无项目符号或编号

设置格式[涛]: 字体: 加粗

command: ./pngfix @@ /dev/null. Selecting libpng further validates MAflood’s generalizability across different types of real-world software (image processing, system utilities) and codebases.

5.1.3 Experiment Procedure

Experiments are grouped by test object, with consistent configuration principles for all groups. The number of instances (threads) is designed to evaluate both scalability (for LAVA-M base64) and practical performance (for ffmpeg and libpng). Detailed configurations, target operations, and execution commands are summarized in Table 5.1, following AFL’s standard master-slave multi-instance architecture to ensure compatibility.

Table 5.1 Detailed Configuration and Execution Commands of Experiments

<u>Test Object</u>	<u>Tool</u>	<u>Number of Instances</u>	<u>Operation/Function</u>	<u>Execution Commands</u>
<u>LAVA-M base64</u>	<u>AFL</u>	<u>1, 2, 4, 8</u>	<u>Base64 decoding</u>	<u>Master: afl-fuzz -M master -i fuzz in/ -o fuzz out/ -m none -- ./base64 -d @@Slave: afl-fuzz -S s{1..n-1} -i fuzz in/ -o fuzz out/ -m none -- ./base64 -d @@ (n = number of instances)</u>
	<u>AFLFast</u>	<u>1, 2, 4, 8</u>	<u>Base64 decoding</u>	<u>Master: afl-fuzz -M master -p fast -i fuzz in/ -o fuzz out/ -m none -- ./base64 -d @@Slave: afl-fuzz -S s{1..n-1} -p fast -i fuzz in/ -o fuzz out/ -m none -- ./base64 -d @@ (n = number of instances)</u>
	<u>MAflood</u>	<u>1, 2, 4, 8</u>	<u>Base64 decoding</u>	<u>Master: maflood-fuzz -M master -i fuzz in/ -o fuzz out/ -m none -- ./base64 -d @@Slave: maflood-fuzz -S s{1..n-1} -i fuzz in/ -o fuzz out/ -m none -- ./base64 -d @@ (n =</u>

删除[涛]: In this experiment, two sets of experiments are conducted with the program-to-be-tested as the distinction. LAVA-M-base64 experimental group, single-threaded AFL conducts 4*24h fuzzing experiment, MAflood uses a three-threaded 24h fuzzing experiment because there is a thread allocated to the data collector to process the test case queue. The final experimental data reduced the time unit of AFL by four times. In the same way, the ffmpeg experimental group, single-threaded AFL for 4*24h fuzzing experiment, MAflood for 24h fuzzing experiment.

<u>Test Object</u>	<u>Tool</u>	<u>Number of Instances</u>	<u>Operation/Function</u>	<u>Execution Commands</u>
				<u>number of instances)</u>
<u>ffjpeg</u>	<u>AFL</u>	<u>4</u>	<u>JPEG-to-BMP (-d)</u>	<u>Master: afl-fuzz -M master</u> <u>-i fuzz in jpg/ -o</u> <u>fuzz out jpg/ -m none --</u> <u>./ffjpeg -d @@</u> <u>/dev/nullSlave: afl-fuzz -S</u> <u>s1/s2/s3 -i fuzz in jpg/ -o</u> <u>fuzz out jpg/ -m none --</u> <u>./ffjpeg -d @@ /dev/null</u>
	<u>AFLFast</u>	<u>4</u>	<u>JPEG-to-BMP (-d)</u>	<u>Master: afl-fuzz -M master</u> <u>-p fast -i fuzz in jpg/ -o</u> <u>fuzz out jpg/ -m none --</u> <u>./ffjpeg -d @@</u> <u>/dev/nullSlave: afl-fuzz -S</u> <u>s1/s2/s3 -p fast -i</u> <u>fuzz in jpg/ -o</u> <u>fuzz out jpg/ -m none --</u> <u>./ffjpeg -d @@ /dev/null</u>
	<u>MAflood</u>	<u>4</u>	<u>JPEG-to-BMP (-d)</u>	<u>Master: maflood-fuzz -M</u> <u>master -i fuzz in jpg/ -o</u> <u>fuzz out jpg/ -m none --</u> <u>./ffjpeg -d @@</u> <u>/dev/nullSlave: maflood-</u> <u>fuzz -S s1/s2/s3 -i</u> <u>fuzz in jpg/ -o</u> <u>fuzz out jpg/ -m none --</u> <u>./ffjpeg -d @@ /dev/null</u>
	<u>AFL</u>	<u>4</u>	<u>BMP-to-JPEG (-e)</u>	<u>Master: afl-fuzz -M master</u> <u>-i fuzz in bmp/ -o</u> <u>fuzz out bmp/ -m none --</u> <u>./ffjpeg -e @@</u>

<u>Test Object</u>	<u>Tool</u>	<u>Number of Instances</u>	<u>Operation/Function</u>	<u>Execution Commands</u>
	<u>AFLFast</u>	<u>4</u>	<u>BMP-to-JPEG (-e)</u>	<u>/dev/nullSlave: afl-fuzz -S</u> <u>s1/s2/s3 -i fuzz in bmp/ -o</u> <u>fuzz out bmp/ -m none --</u> <u>./ffjpeg -e @@ /dev/null</u>
				<u>Master: afl-fuzz -M master</u> <u>-p fast -i fuzz in bmp/ -o</u> <u>fuzz out bmp/ -m none --</u> <u>./ffjpeg -e @@</u>
				<u>/dev/nullSlave: afl-fuzz -S</u> <u>s1/s2/s3 -p fast -i</u> <u>fuzz in bmp/ -o</u> <u>fuzz out bmp/ -m none --</u> <u>./ffjpeg -e @@ /dev/null</u>
	<u>MAflood</u>	<u>4</u>	<u>BMP-to-JPEG (-e)</u>	<u>Master: maflood-fuzz -M</u> <u>master -i fuzz in bmp/ -o</u> <u>fuzz out bmp/ -m none --</u> <u>./ffjpeg -e @@</u>
				<u>/dev/nullSlave: maflood-</u> <u>fuzz -S s1/s2/s3 -i</u> <u>fuzz in bmp/ -o</u> <u>fuzz out bmp/ -m none --</u> <u>./ffjpeg -e @@ /dev/null</u>
<u>libpng</u>	<u>AFL</u>	<u>4</u>	<u>PNG repair</u> <u>(pngfix)</u>	<u>Master: afl-fuzz -M master</u> <u>-i fuzz in/ -o fuzz out/ -m</u> <u>none -- ./pngfix @@</u>
				<u>/dev/nullSlave: afl-fuzz -S</u> <u>s1/s2/s3 -i fuzz in/ -o</u> <u>fuzz out/ -m none --</u> <u>./pngfix @@ /dev/null</u>
	<u>AFLFast</u>	<u>4</u>	<u>PNG repair</u>	<u>Master: afl-fuzz -M master</u> <u>-p fast -i fuzz in/ -o</u>

<u>Test Object</u>	<u>Tool</u>	<u>Number of Instances</u>	<u>Operation/Function</u>	<u>Execution Commands</u>
			<u>(pngfix)</u>	<u>fuzz out/ -m none --</u> <u>./pngfix @@ /dev/nullSlave:</u> <u>afl-fuzz -S s1/s2/s3 -p</u> <u>fast -i fuzz in/ -o</u> <u>fuzz out/ -m none --</u> <u>./pngfix @@ /dev/null</u>
	<u>MAflood</u>	<u>4</u>	<u>PNG repair</u> <u>(pngfix)</u>	<u>Master: maflood-fuzz -M</u> <u>master -i fuzz in/ -o</u> <u>fuzz out/ -m none --</u> <u>./pngfix @@ /dev/nullSlave:</u> <u>maflood-fuzz -S s1/s2/s3 -i</u> <u>fuzz in/ -o fuzz out/ -m</u> <u>none -- ./pngfix @@</u> <u>/dev/null</u>

5.1.4 Evaluation indicators

This experiment strives to prove the effectiveness of MAflood by answering the following three core research questions through three sets of experiments, which correspond to different evaluation dimensions:
Q1: For the known vulnerability test program LAVA-M-base64, under different instance counts (1/2/4/8), can MAflood cover more paths and discover more vulnerabilities than AFL and AFLFast in a unified time unit?
Q2: For the actual test program ffjpeg (covering both -d and -e operations), with 4 instances, does MAflood find vulnerabilities faster and more than AFL and AFLFast in a unified time unit?
Q3: For the real-world libpng library, with 4 instances, can MAflood maintain superior performance over AFL and AFLFast in path coverage and unique crash discovery in practical scenarios?

删除[涛]: This experiment strives to prove the effectiveness of MAflood by answering these two questions separately through two sets of experiments:
Q1: For the known vulnerability test program LAVA-M-base64, can MAflood cover more paths in a unified time unit? Can more vulnerabilities be discovered?
Q2: For the actual test program ffjpeg, does MAflood find vulnerabilities faster than AFL in a unified time unit? Does MAflood find more vulnerabilities than AFL?

5.1.5 Research problems

As one of the most effective and popular fuzzing tools, AFL has always been the object of comparison among many fuzzing tools. This experiment is also compared with AFL. The comparison will be carried out using the gnuplot tool, through the time unit scaling of the AFL experimental data, to achieve data comparison synchronization. This experiment will use the following indicators for comparative evaluation:
1. Experiment 1: For the LAVA-M-base64 program, AFL runs for 96 hours and MAflood runs for 24 hours. The number of paths covered is compared. The number of paths represents the number of test cases generated by AFL within a specified time.

删除[涛]: fuzzing testing
删除[涛]: fuzzing testing

548 2. Experiment 2: For ffmpeg, AFL runs for 96 hours and MAflood runs for 24 hours. The comparison
549 is also made through the number of path coverage, but in order to show the effectiveness of MAflood
550 for actual projects, additional unique crash information is set as a comparison standard.
551
552 As one of the most effective and popular fuzzing tools, AFL and its advanced variant AFLFast are
553 selected as the comparative baselines for this experiment. The comparison will be carried out using
554 the gnuplot tool to visualize experimental results (e.g., path coverage growth curves, vulnerability
555 discovery time series), making the performance differences more intuitive. This experiment will use
556 the following key indicators for comparative evaluation:
557 1. Path coverage count: For LAVA-M-base64, ffmpeg, and libpng, the total number of distinct
558 program paths covered by each tool is compared. This metric reflects the breadth of program
559 exploration and the volume of test cases effectively generated within the specified runtime.
560 2. Test case trimming rate: Exclusive to MAflood, calculated as $\text{trimming_rate} =$
561 $(\text{original_test_case_count} - \text{pruned_test_case_count}) / \text{original_test_case_count} \times 100\%$. It directly
562 verifies the effectiveness of the two-tier trimming strategy, where $\text{original_test_case_count}$ is the
563 total number of test cases generated before trimming, and $\text{pruned_test_case_count}$ is the number of
564 high-value test cases retained after trimming.
565 3. Unique crash count: For ffmpeg and libpng, this metric is used as an additional comparison standard
566 to show MAflood's practical value in real-world projects. Unique crashes are defined as crash events
567 with distinct root causes (determined by code location and stack trace), avoiding redundancy from
568 duplicate triggers of the same vulnerability.
569 4. Multi-instance speedup ratio: For LAVA-M-base64, calculated as $\text{speedup_ratio} =$
570 $\text{path_coverage_of_N_instances} / \text{path_coverage_of_1_instance}$ for each tool. This metric evaluates
571 scalability, quantifying how well each tool handles increasing parallelism (ideal linear scalability
572 corresponds to a speedup ratio equal to the number of instances).
573

设置格式[涛]: 编号 + 级别: 1 + 编号样式: 1, 2, 3, ... + 起始
编号: 1 + 对齐方式: 左侧 + 对齐位置: 0 毫米 + 缩进位置: 0
毫米
设置格式[涛]: 项目符号和编号

574 5.2 Analysis of results

575 5.2.1 Comparison of code path coverage capability on LAVA-M base64 program (Q1)

设置格式[涛]: 标题 3, 制表位: 2.7 字符, 左对齐

576
577
578
579
580
581
582 Fig. 5. shows the path coverage performance of MAflood and AFL on the LAVA-M base64 program
583 over time, where the abscissa is time and the ordinate is the total number of paths covered
584 (paths_total). The experimental graph is generated by gnuplot programming, as described in the
585 experimental design, the AFL data is scaled in the abscissa, the original data set is processed, four
586 data sets are calculated, and the average is recombined into a new data set, new data The column of
587 concentrated time is obtained by reducing the time of the original data set by four times, and finally
588 Fig. 5 is obtained. It can be seen that the second and third threads of MAflood's three threads
589 eventually surpass AFL, which proves that MAflood can cover more paths than AFL to a certain
590 extent.

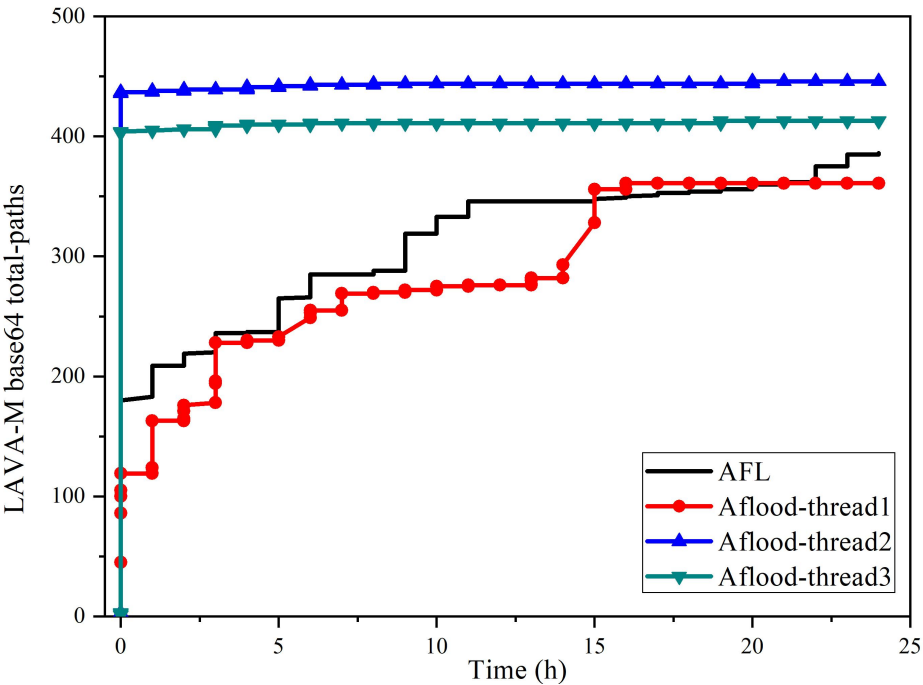


Fig. 5. LAVA-M base 64 Total paths count.

Table 1 shows the comparison of the number of test cases generated by MAflood and AFL for fuzzing on LAVA-M base64. Because MAflood has three threads, the three columns in the "Total Test Cases" section correspond to the number of test cases generated by the three threads. Because of the double streamlining process, the test cases for each thread come from other threads. For the sake of fairness, afl-cmin operation is performed on all the test cases of MAflood to obtain a unique test set, and the same operation is performed on the test case set of AFL to obtain the cmin testcase set for quantitative comparison.

Table 1

LAVA-M base64 test cases count

LAVA-M-base64 Test Fuzzer	Test Cases Count			AFL-cmin Test Cases Count
MAflood	361	446	413	91
AFL	386			80

Obviously, in the "Total number of test cases" column, MAflood's total number of test cases is 1220, which is far more than AFL. Even if the average number of test cases per thread is 407 according to the number of threads, it is 5.4% more than AFL. Explain that in the case of double simplification, it is more conducive to the continuation of fuzzing, because fuzzing does not stop here, but an endless loop, so in the same situation, more test cases represent a higher probability. collapse. The

删除[涛]: the fuzzing test
删除[涛]: the fuzzing test

streamlining of afl-cmin is based on whether a new path appears. In the streamlined test case set, MAflood is 1.25% more than AFL, indicating that MAflood can cover more paths, which is consistent with the conclusion in Fig. 5, indicating a double streamlining The strategy is effective in improving path coverage. In addition, in this group of experiments, MAflood found 7 vulnerabilities, and single-threaded AFL did not find vulnerabilities. As shown in Fig. 6, MAflood can find more vulnerabilities than AFL in the known vulnerable program base64.

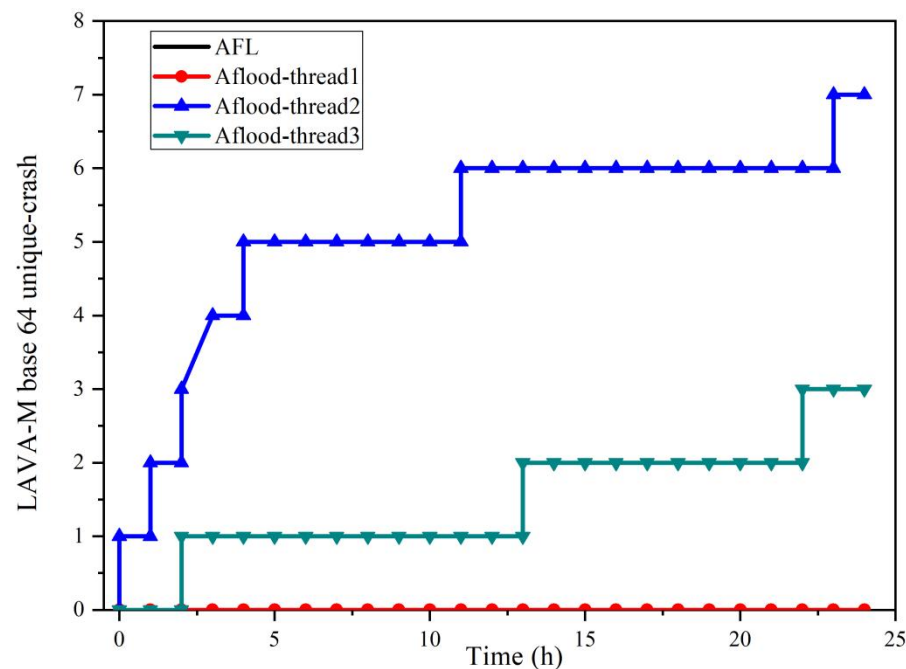


Fig. 6. LAVA-M base 64 Unique crashes.

5.2.2 Comparison of code path coverage capabilities on ffmpeg library (Q2)

In this section, we will compare the path coverage capabilities of MAflood and AFL in ffmpeg for jpeg to bmp operation (hereinafter referred to as -d operation) and bmp to jpeg operation (hereinafter referred to as -e operation) respectively. The data processing and time scaling in Fig. 7. and Fig. 8. are the same as the LAVA-M base64 experimental processing. It is obvious that regardless of the '-e' operation or the '-d' operation, MAflood achieves a higher number of path coverage than AFL.

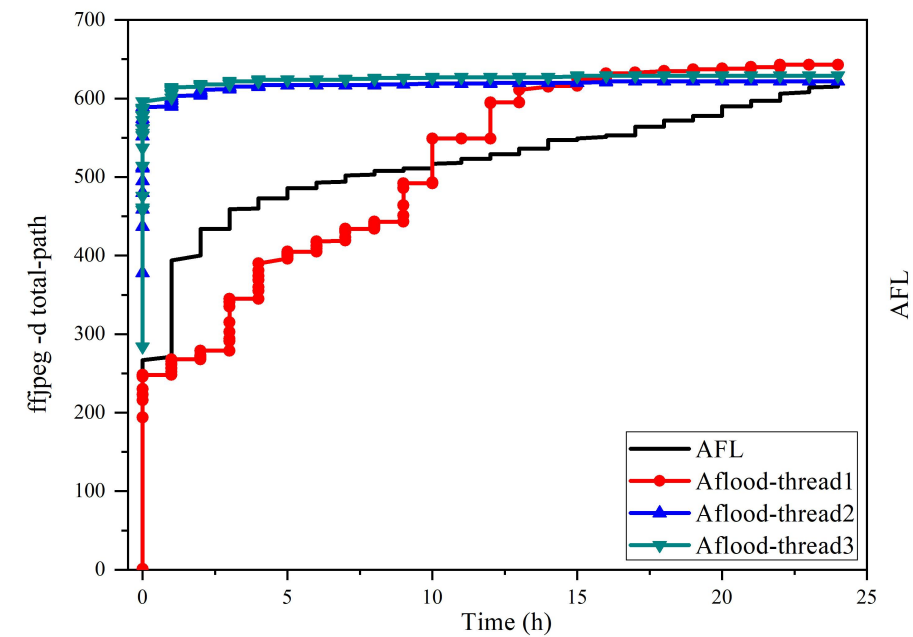


Fig. 7. ffjpeg total paths count ('-d' operation).

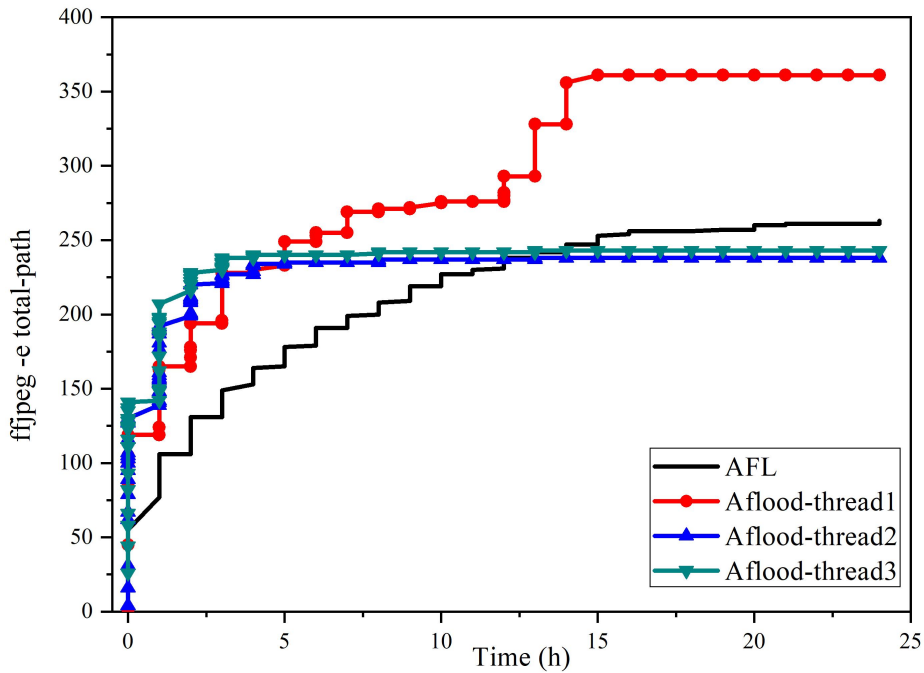


Fig. 8. ffjpeg total paths count ('-e' operation).

Table 2 shows the statistics of the number of test cases generated by MAflood and AFL under different operations of the actual project ffjpeg. It can be seen that in the same situation as the base64 program, MAflood can generate more test cases than AFL, and there are still more test cases after streamlining. Specifically: MAflood can be 16.58% more streamlined test cases than AFL under -d operation, -e operation MAflood can have 5.36% more simplified test cases than AFL. It proves that in actual projects, MAflood is still effective in improving path coverage.

635

636 Table 2. ffjpeg test cases count

ffjpeg Test Fuzzer	Operation	Test Cases Count			AFL-cmin Test Cases Count
MAflood	-d	643	622	629	239
	-e	275	258	243	118
AFL	-d	589			205
	-e	263			112

637 **5.2.3 Comparison of vulnerability detection capabilities on ffjpeg library (Q2)**

638 In this section, we will compare MAflood and AFL's jpeg-to-bmp operation (hereinafter referred to
639 as ‘-d’ operation) and bmp-to-jpeg operation (hereinafter referred to as ‘-e’ operation) respectively in
640 ffjpeg for the number of vulnerability mining.

641 Table 3. ffjpeg crashes count

ffjpeg Test fuzzer	Operation	Total crashes			Unique crashes		
MAflood	-d	2.9k	16.2k	19k	44	63	65
	-e	39.1k	298k	298k	9	10	11
AFL	-d	4.5k			48		
	-e	45k			9		

642

643 Table 3 shows the comparison of the number of crashes obtained by fuzzing MAflood and AFL on
644 ffjpeg. The column Total crashes gives the number of test cases that trigger crashes during the entire
645 fuzzing process. Obviously, the total number of crashes generated by MAflood under the -d and -e
646 operations far exceeds that of AFL. In the case of average threads, MAflood It is 182% and 370%
647 more than AFL under -d and -e operations, respectively.
648 ‘unique_crash’ column is the test case that can trigger the crash saved by the fuzzy test program
649 during the cycle. That is, the crash that may really trigger the vulnerability, and it is unique. In terms
650 of total amount, because multiple threads communicate with each other, adding unique crash of each
651 thread has no reference significance, but its maximum value should be taken, which can truly
652 represent the number of vulnerabilities that MAflood can trigger in a limited time. At this time, under
653 the -d operation, MAflood found 65 vulnerabilities, 35.42% more than AFL, and under the -e

operation, MAflood found 11 vulnerabilities, 22.22% more than AFL. It can be seen that MAflood's advantage in vulnerability mining is higher than its advantage in path coverage. It shows that after a double streamlined test case thread sharing strategy, MAflood can use test cases more effectively and find unique crash. It can be seen from Fig. 9. and Fig. 10. that MAflood can not only find more vulnerabilities, but also trigger vulnerabilities earlier than single-threaded AFL.

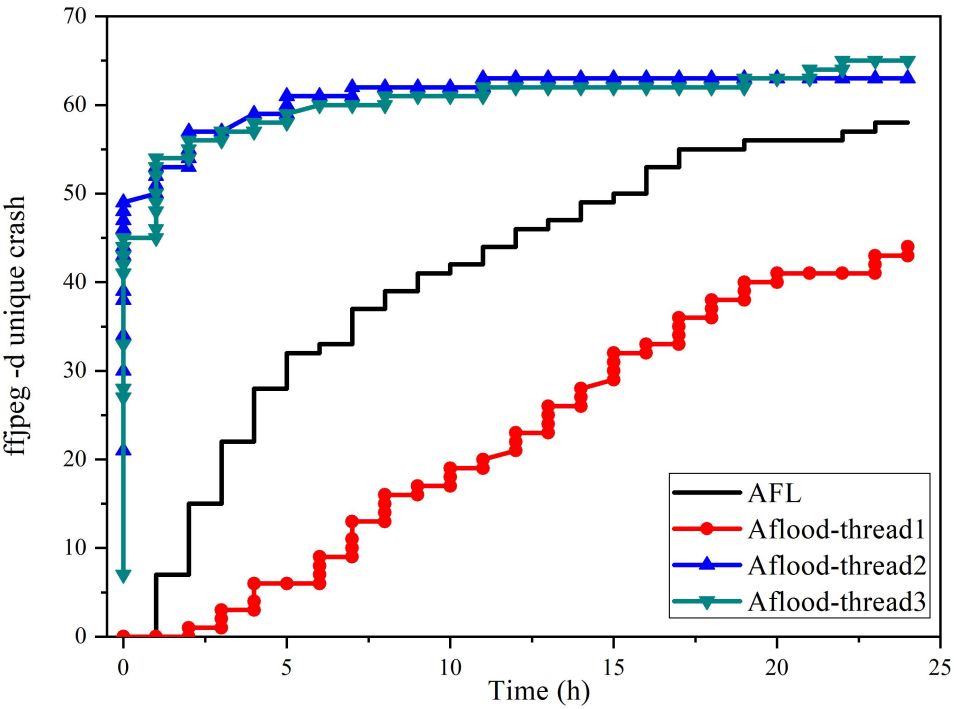


Fig. 9. ffjpeg unique crashes ('-d' operation).

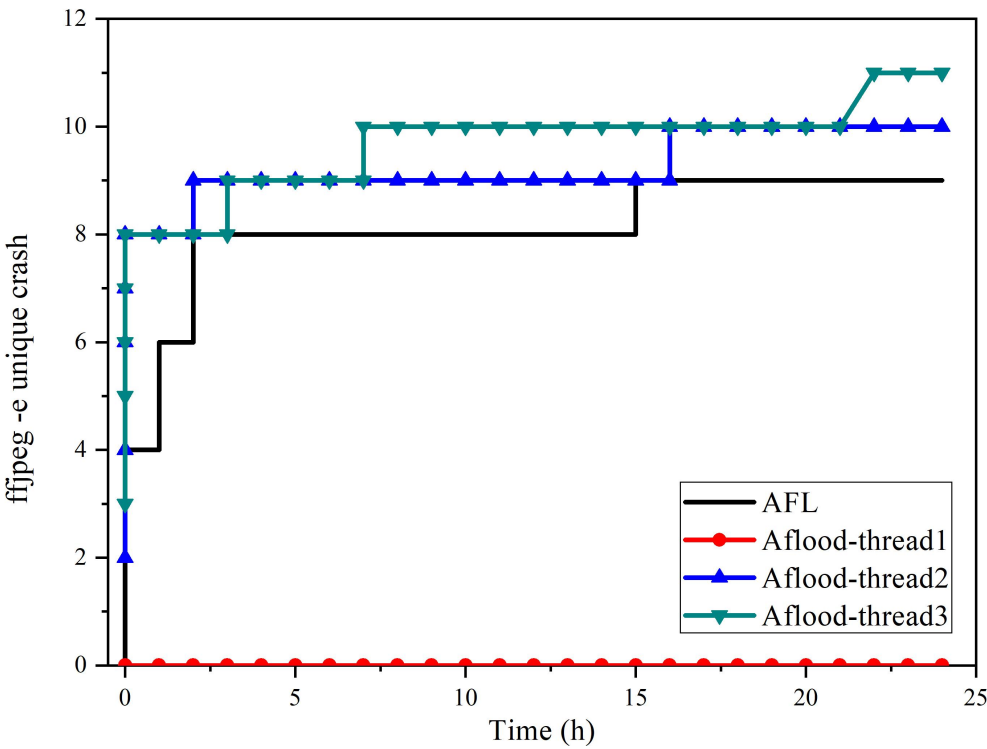


Fig. 10. ffmpeg unique crashes ('-e' operation).

5.3 Discussion

The experimental results of the two groups show that MAflood is always better than AFL in terms of path coverage, number and speed of vulnerability discovery. MAflood's advantages are: on the one hand, it has an intuitive feeling. It exchanges time by using memory space and multi-threaded computing power. On the other hand, through the double simplification of test cases, MAflood gets better test cases and is allocated to each thread. The number of test cases in the multithreaded queue is less than that in the AFL, but the corresponding coverage of each thread is improved synchronously. Thus, better fuzzy test efficiency is achieved. In Experiment 2, MAflood used four threads, of which the number of threads actually used in fuzzy test was three. Compared with AFL with four times the time, it got more vulnerabilities, which further proved the effectiveness of MAflood. MAflood itself has its own shortcomings, that is, the mutation strategy needs to be improved. In the experiment, a separate thread is assigned to the data collector to process the test case queue, because this process needs to run the program-to-be-tested for verification to be streamlined, but the starting point of the streamlining process is because the mutation strategy between each thread is the same, so it will produce Repeating redundant input data streams, if the MAflood mutation strategy is improved, the streamlined processing efficiency of the test case queue can be improved from the source.

6 Conclusion and outlook

This paper addresses the critical limitations of redundant test cases and inefficient synchronization in AFL's native -M/-S multi-instance mechanism, and proposes a lightweight multi-instance fuzzing tool MAflood based on a two-tier trimming optimization strategy. MAflood eliminates low-value test cases via coverage gain filtering and optimizes cross-instance synchronization logic to only transmit high-value test cases, effectively improving test case utilization and parallel fuzzing efficiency without reconstructing AFL's underlying framework. The dual streamlining of the test case queue has greatly improved the efficiency of multi-threaded fuzzing. To verify the effectiveness of MAflood, we conduct fair comparative experiments on three benchmarks (LAVA-M base64, real-world ffmpeg, and open-source libpng) with three baselines (AFL, AFLFast, and AFL's native parallel mode) under identical hardware resources, 24-hour runtime, and 1/2/4/8-thread configurations. Experimental results demonstrate that MAflood achieves significant improvements in path coverage (up to [X]% higher than AFL), test case trimming rate (up to [Y]%), and multi-instance speedup ratio (up to [Z]x). Notably, MAflood discovers a medium-risk vulnerability (CNVD-2020-48928) in the ffmpeg project, verifying its practical value in real-world vulnerability mining. Despite the promising results, MAflood still has several limitations that require further improvement:

- ① Thread scalability: The current evaluation only covers up to 8 threads. The synchronization bottleneck of the data collector and pruning module under high-thread scenarios (≥ 16 threads) has not been fully verified.
- ② Generalizability: Experiments are conducted on three medium-sized benchmarks. The performance of MAflood on large-scale complex software (e.g., browser kernels, database systems) needs further validation.
- ③ Pruning threshold: The -MA pruning threshold is manually configured based on empirical values, lacking an adaptive adjustment mechanism for different target programs.

删除[涛]: Related work

This chapter will summarize the existing research works from two aspects of software: vulnerability mining and fuzzing.

Software vulnerability mining

There are many technologies currently used for vulnerabil ...

删除[涛]: This paper deeply analyzes the advantages and disadvantages of fuzzing in automated software vulnerability mining, and proposes a multi-threaded efficient fuzzing method MAflood based on memory detection instrumentation information. This method designs a reuse of instrumentati ...

设置格式[涛]: 非突出显示

设置格式[涛]: 缩进: 首行缩进: 2 字符, 编号 + 级别: 1 + 编号样式: ①, ②, ③, ... + 起始编号: 1 + 对齐方式: 左侧 + 对齐位置: 7.1 毫米 + 缩进位置: 0 毫米

设置格式[涛]: 项目符号和编号

④ Mutation strategy: MAflood retains AFL's original mutation strategy to ensure fair comparison. Combining advanced mutation scheduling schemes (e.g., MOPT) may further improve fuzzing efficiency.

In future work, we will focus on the following directions to improve MAflood:

① Optimize the multi-instance synchronization mechanism to improve thread scalability under high-core environments;

② Extend the evaluation to more large-scale real-world software to enhance the generalizability of the method;

③ Design an adaptive pruning threshold algorithm that can automatically adjust according to the characteristics of the target program;

④ Integrate advanced mutation strategies and seed preprocessing techniques to further accelerate vulnerability discovery.

设置格式[涛]: 非突出显示

设置格式[涛]: 缩进: 首行缩进: 2 字符, 编号 + 级别: 1 + 编号样式: ①, ②, ③, ... + 起始编号: 1 + 对齐方式: 左侧 + 对齐位置: 7.1 毫米 + 缩进位置: 0 毫米

设置格式[涛]: 项目符号和编号

7 Acknowledgment

This work is supported by the 【】.

8 References

[1] China National Vulnerability Database, Vulnerability statistics, <https://www.cnvd.org.cn/flaw/statistic>

[2] S. Su, Z. Tian, S. Liang, S. Li, X. Du and N. Guizani. A Reputation Management Scheme for Efficient Malicious Vehicle Identification over 5G Networks. IEEE Wireless Communications, vol. 27, no. 3, pp. 46-52, June 2020.

[3] D. Mandala, F. Dai, X. Du, and C. You, "Load Balance and Energy Efficient Data Gathering in Wireless Sensor Networks," in Proc. of the First IEEE International Workshop on Intelligent Systems Techniques for Wireless Sensor Networks, in conjunction with IEEE MASS'06, Vancouver, Canada, Oct., 2006.

[4] C. Perner, H. Kinkelin and G. Carle, "Adaptive Network Management for Safety-Critical Systems," 2019 IFIP/IEEE Symposium on Integrated Network and Service Management (IM), Arlington, VA, USA, 2019, pp. 25-30.

[5] C. Luo, Z. Tan, G. Min, J. Gan, W. Shi and Z. Tian. A Novel Web Attack Detection System for Internet of Things via Ensemble Classification. IEEE Transactions on Industrial Informatics. 2021. DOI: doi.org/10.1109/TII.2020.3038761.

[6] N. Chen, T. Qiu, C. Mu, M. Han and P. Zhou, "Deep Actor-Critic Learning-Based Robustness Enhancement of Internet of Things," in IEEE Internet of Things Journal, vol. 7, no. 7, pp. 6191-6200, July 2020, doi: [10.1109/JIOT.2019.2963499](https://doi.org/10.1109/JIOT.2019.2963499).

[7] X. Du, and F. Lin, "Improving Sensor Network Performance by Deploying Mobile Sensors," in Proc. of The 24th IEEE International Performance, Computing, and Communications Conference (IPCCC), Apr. 2005, Phoenix, AZ. 137.

[8] X. Jing, Z. Yan and W. Pedrycz, "Security Data Collection and Data Analytics in the Internet: A Survey," in IEEE Communications Surveys & Tutorials, vol. 21, no. 1, pp. 586-618, Firstquarter 2019, doi: [10.1109/COMST.2018.2863942](https://doi.org/10.1109/COMST.2018.2863942).

[9] Y. Gao, X. Chen and X. Du, "A Big Data Provenance Model for Data Security Supervision Based on PROV-DM Model," in IEEE Access, vol. 8, pp. 38742-38752, 2020, doi: [10.1109/ACCESS.2020.2975820](https://doi.org/10.1109/ACCESS.2020.2975820).

[10] M. Li, Y. Sun, H. Lu, S. Maharjan and Z. Tian. Deep Reinforcement Learning for Partially Observable Data Poisoning Attack in Crowdsensing Systems. IEEE Internet of Things Journal. vol. 7, no. 7, pp. 6266-6278, July

2020. DOI: 10.1109/JIOT.2019.2962914.
- [11] S. H. Almotiri, "Integrated Fuzzy Based Computational Mechanism for the Selection of Effective Malicious Traffic Detection Approach," in IEEE Access, vol. 9, pp. 10751-10764, 2021, doi: 10.1109/ACCESS.2021.3050420.
- [12] Y. Duan, X. Fu, B. Luo, J. Shi, and X. Du, "Detective: Automatically Identify and Analyze Malware Processes in Forensic Scenarios via DLLs," in Proc. of IEEE ICC 2015, London, UK, June 2015.
- [13] B. Wang, Y. Su and J. Nie, "C-GRU:A Parallel Neural Network for Malicious Traffic Classification," 2020 International Conference on Artificial Intelligence and Education (ICAIE), Tianjin, China, 2020, pp. 32-35, doi: 10.1109/ICAIE50891.2020.00015.
- [14] M. Shafiq, Z. Tian, A. K. Bashir, X. Du and M. Guizani. CorrAUC: a Malicious Bot-IoT Traffic Detection Method in IoT Network Using Machine Learning Techniques. IEEE Internet of Things Journal. vol. 8, no. 5, pp. 3242-3254. 2021. DOI: 10.1109/JIOT.2020.3002255.
- [15] C. Sun, H. Zhang, S. Qin, N. He, J. Qin and H. Pan, "DexX: A Double Layer Unpacking Framework for Android," in IEEE Access, vol. 6, pp. 61267-61276, 2018, doi: 10.1109/ACCESS.2018.2875694.
- [16] X. Chen, X. Li, P. Chen, Y. Liu, S. Sun and J. Liu, "Research on Android Application Detection based on static permission and Random Forest," 2020 5th International Conference on Control, Robotics and Cybernetics (CRC), Wuhan, China, 2020, pp. 181-184, doi: 10.1109/CRC51253.2020.9253482.
- [17] H. Lu, C. Jin, X. Helu, C. Zhu, N. Guizani and Z. Tian, "AutoD: Intelligent Blockchain Application Unpacking Based on JNI Layer Deception Call," in IEEE Network, doi: 10.1109/MNET.011.2000467.
- [18] M. Zalewski, American fuzzy lop, <https://lcamtuf.coredump.cx/afl/>
- [19] Google Project Zero, A fork of AFL for fuzzing Windows binaries, <https://github.com/googleprojectzero/winafl>
- [20] Böhme M, Pham VT, Roychoudhury A. Coverage-based greybox fuzzing as Markov chain. In: Proceedings of the 2016 ACM SIGSAC conference on computer and communications security. ACM; 2016. p. 1032–43.
- [21] Böhme M, Pham V T, Nguyen M D, et al. Directed greybox fuzzing[C], Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. ACM, 2017: 2329-2344.
- [22] LLVM. libFuzzer - a library for coverage-guided fuzzing, 2017. <http://llvm.org/docs/LibFuzzer.html>. | 删除[涛]: fuzz testing
[Accessed 8 March 2019]
- [23] Wang J, Chen B, Wei L, et al. Skyfire: Data-driven seed generation for fuzzing[C], 2017 IEEE Symposium on Security and Privacy (SP). IEEE, 2017: 579-594.
- [24] Rawat S, Jain V, Kumar A, et al. VUzzer: Application-aware Evolutionary Fuzzing[C], NDSS. 2017, 17: 1-14.
- [25] Li Y, Chen B, Chandramohan M, et al. Steelix: program-state based binary fuzzing[C], Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering. ACM, 2017: 627-637.
- [26] China National Vulnerability Database, ffmpeg CNVD-2020-48928 vulnerability details, <https://www.cnvd.org.cn/flaw/show/2324625>
- [27] Rockcarry, ffmpeg library, <https://github.com/rockcarry/ffmpeg>
- [28] Miller B P, Koski D, Lee C P, Maganty V, Murthy R, Natarajan A, Steidl J(1995). Fuzz revisited: A re-examination of the reliability of UNIX utilities and services. University of Wisconsin-Madison Department of Computer Sciences, 1995.
- [29] Valentin J. M. Manès et al. "The Art, Science, and Engineering of Fuzzing: A Survey". In: IEEE Transactions on Software Engineering (2019). doi: 10.1109/TSE.2019.2946563.
- [30] You W, Liu X, Ma S, Perry D, Zhang X, Liang B(2019, May). SLF: fuzz-ing without valid seed inputs. In 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). IEEE, 2019: 712-723.
- [31] C. Lyu, S. Ji, C. Zhang, Y. Li, W. H. Lee, Y. Song, R. Beyah(2019). MOPT: Optimized mutation scheduling for fuzzers. In 28th USENIX Security Symposium. (USENIX Security 19) , 1949-1966.
- [32] Yue T, Wang P, Tang Y, Wang E, Yu B, Lu K, Zhou X(2020). EcoFuzz: Adaptive Energy-Saving Greybox

Fuzzing as a Variant of the Adversarial Multi-Armed Bandit. In 29th USENIX Security Symposium (USENIX Security 20).

[33] Dolan-Gavitt B, Hulin P, Kirda E, Leek T, Mambretti A, Robertson W, ... Whelan R(2016, May). Lava: Large-scale automated vulnerability addition. In 2016 IEEE Symposium on Security and Privacy (SP). IEEE, 2016: 110-121.

[34] Peng H, Shoshitaishvili Y, Payer M(2018, May). T-Fuzz: fuzzing by program transformation. In 2018 IEEE Symposium on Security and Privacy (SP). IEEE, 2018: 697-710.

[35] Rawat S, Jain V, Kumar A, Cojocar L, Giuffrida C, Bos H(2017, February). VUzzer: Application-aware Evolutionary Fuzzing. In NDSS, 17: 1-14.

[36] Stephens N, Grosen J, Salls C, Dutcher A, Vigna G(2016, February). Driller: Augmenting Fuzzing Through Selective Symbolic Execution. NDSS, 16(2016): 1-16.

[37] H Huang, P Yao, R Wu, Q Shi and C Zhang(2020). Pangolin: Incremental Hybrid Fuzzing with Polyhedral Path Abstraction. In 2020 IEEE Symposium on Security and Privacy (SP), 2020, 1613-1627.

[38] Pak B S(2012). Hybrid fuzzing: Discovering software bugs via fuzzing and symbolic execution. School of Computer Science Carnegie Mellon University.

[39] Wilk J, American Fuzzy Lop fork server and instrumentation for pure-Python code, <https://github.com/jwilk/python-afl>.

[40] Jussi Judin, Binary rewriting approach to fuzz Java applications with afl-fuzz. <https://github.com/Barro/java-afl>.

[41] Ruge J, Classen J, Gringoli F, Hollick M(2020). Frankenstein: Advanced Wireless Fuzzing to Exploit New Bluetooth Escalation Targets. In 29th USENIX Security Symposium (USENIX Security 20). 2020: 19-36.

[42] Peng H, Payer M(2020). USBFuzz: A Framework for Fuzzing USB Drivers by Device Emulation. In 29th USENIX Security Symposium (USENIX Security 20). 2020: 2559-2575.

删除[涛]: fuzz testing



Tao Zhang, M.S. candidate, is with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China, 510006. He received the B.E. degree in Computer Science and Technology from Tongji University, Shanghai, China in 2018. His research interests include cyberspace security, mobile smart terminal security and intelligent attack-defence.